Data Representation

Michael Cao

Topic 1: User-defined Data Types

Michael Cao

Learning Objective

In this lesson, you will learn:

- User-defined data types

13.1 User-defined data types	
Candidates should be able to:	Notes and guidance
Show understanding of why user-defined types are necessary	
Define and use non-composite types	Including enumerated, pointer
Define and use composite data types	Including set, record and class/object
Choose and design an appropriate user-defined data type for a given problem	

Enumerated Data Type

Non-composite data type can be defined without referencing another data type:

- Enumerated data type: defined by a given list of all possible values that has an implied order
- It looks like a string but without quotation. It is just a type of user-defined type

```
TYPE <identifier> = (value1, value2, value3, ... )
```

```
TYPE Tmonth = (January, February, March,
April, May, June, July, August, September,
October, November, December)
```

```
DECLARE thisMonth : Tmonth
DECLARE nextMonth : Tmonth
thisMonth ← January
nextMonth ← thisMonth + 1
```

Pointer Data Type

Pointer is another non-composite user-defined data type:

- Used to reference a memory location
- Use ^ to reference as well as dereference

```
TYPE <pointer> = ^<Typename>
```

```
TYPE TIntPt = ^INTEGER
DECLARE MyIntPt: TIntPt
DECLARE Num1, Num2: INTEGER
```

```
Num1 ← 100
MyIntPt ← ^Num1
Num2 ← MyIntPt^ * 2
```

Composite Data Type

A composite data type refers to other data type in its definition:

- Record: introduced in AS
- Class: include variables of given data types and methods
- Set: a given list of unordered elements that can use set of theory operations

Set

A set is a given list of unordered elements that can use set of theory operations

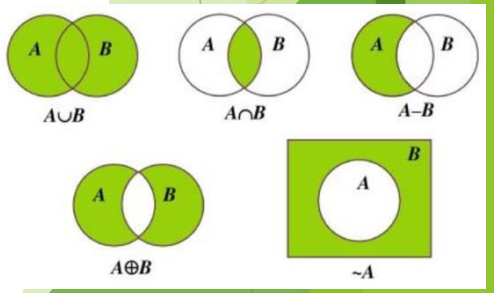
- Duplicated values are not allowed
- No organization of the data
- Operations include:
  - Check existence, add new values, remove existing v
  - Add and subtract between sets
  - Intersection, union and complement

```
TYPE <identifier1> = SET OF <data type>
DEFINE <identifier2> (value1, value2, value, … ) : <identifier1>
```

Example – declaration of Set data type

This user-defined data type holds data about vowels.

```
TYPE LetterSet = SET OF CHAR
DEFINE Vowels ('A','E','I','O','U'): LetterSet
```



Activities

Please follow the steps to enhance the understanding of different data types:

- Read more codes on textbook and pseudocode guide
- Do exercise on handouts
- Implement them on python (enumerate, set)
- Explore: Other composite or user-defined data type?
- Discuss: The need for different type of data type above?
- Finish the homework

Topic 2: Floating-Point

Michael Cao

Learning Objective

In this lesson, you will learn:

- The representation and manipulation of float-point numbers

13.3 Floating-point numbers, representation and manipulation	
Candidates should be able to:	Notes and guidance
Describe the format of binary floating-point real numbers	Use two's complement form Understand of the effects of changing the allocation of bits to mantissa and exponent in a floating-point representation
Convert binary floating-point real numbers into denary and vice versa	
Normalise floating-point numbers	Understand the reasons for normalisation
Show understanding of the consequences of a binary representation only being an approximation to the real number it represents (in certain cases)	Understand how underflow and overflow can occur
Show understanding that binary representations can give rise to rounding errors	

Floating-Point Number

We use two's complement to represent signed numbers, but this limit the range of numbers and cannot indicate fractional values.

Instead, we use scientific notation in binary number system

where M is Mantissa and E is the Exponent $M \times 2^E$

Guess: Why we c

–1	•	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{8}$	$\frac{1}{16}$	$\frac{1}{32}$	$\frac{1}{64}$	$\frac{1}{128}$	–128	64	32	16	8	4	2	1

Binary to Denary

- The binary numbers are all in two's complement
- There are generally 2 methods for converting, choose the proper one
- Use the right as an example
- Practice from easy to difficult from example 13.1 to 13.4 on Hodder textbook

-1	•	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{8}$	$\frac{1}{16}$	$\frac{1}{32}$	$\frac{1}{64}$	$\frac{1}{128}$
0		1	0	1	1	0	1	0

-128	64	32	16	8	4	2	1
0	0	0	0	0	1	0	0

Solution

Method 1

Add up the mantissa values where a 1 bit appears:

$$M = \frac{1}{2} + \frac{1}{8} + \frac{1}{16} + \frac{1}{64} \equiv \frac{32}{64} + \frac{8}{64} + \frac{4}{64} + \frac{1}{64} = \frac{45}{64}$$

Add up the exponent values where a 1 bit appears:

$E = 4$

Use $M \times 2^E$:

$$\frac{45}{64} \times 2^4 = \frac{45}{64} \times 16$$
$$= 0.703125 \times 16$$
$$= 11.25 \text{ (the denary value)}$$

Method 2

Write the mantissa as 0.1011010.

The exponent is 4, so move the binary point four places to the right (to match the exponent value). This gives 01011.010.

whole number part						**fraction part**		
-16	8	4	2	1	•	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{8}$
0	1	0	1	1		0	1	0

This gives 11.25 (the same result as method 1).

Denary to Binary

- There are still 2 methods for converting, choose the proper one
- For normalization, start with 0.1 if positive and 1.0 with negative
- Practice from example 13.5 to 13.7 (use 13.5 as an example)
- Use approximate value if cannot make it so accurate (take 5.88 as example)

To convert this into binary, we will use a method similar to that used in Chapter 1.

.88 × 2 = 1.76 so we will use the 1 value to give 0.1

.76 × 2 = 1.52 so we will use the 1 value to give 0.11

.52 × 2 = 1.04 so we will use the 1 value to give 0.111

.04 × 2 = 0.08 so we will use the 0 value to give 0.1110

.08 × 2 = 0.16 so we will use the 0 value to give 0.11100

.16 × 2 = 0.32 so we will use the 0 value to give 0.111000

.32 × 2 = 0.64 so we will use the 0 value to give 0.1110000

.64 × 2 = 1.28 so we will use the 1 value to give 0.11100001

We have to stop here since our system uses a maximum of 8 bits. Now the value of 5 (in binary) is 0101; this gives:

5.88 ≡ 0101.11100001

Moving the binary point as far to the left as possible gives:

0.1011100×2^3 (2^3 since the binary point was moved three places).

Thus, we get 0.1011100 00000011
(mantissa) (exponent)

Potential Problems

During the calculation process, please summarize the potential problems before we give the conclusion

- Rounding error: cannot convert the denary number into an exactly binary equivalent
- Precision vs. Range: trade-off in limited bit length
- Overflow and Underflow: result is too big or small the represent
- Zero cannot be represented